Mojo-V ISA Extension Specification

Document version: 1.01, Mojo-V specification version (`mojov_ver`) #0x1
Release date: July 21, 2026.
To provide feedback: Email: mojov-devs@umich.edu

NOTE: The latest version of this document is here: https://github.com/toddmaustin/mojo-v

1. Introduction

Mojo-V (pronounced “mojo-five”) extends the RISC-V instruction architecture to provide **secret computation** that is **blind**—secrets can’t be viewed directly—and **silent**—secrets can’t be inferred through side channels. It provides cryptographically enforced privacy with minimal performance overhead and enables processors to execute operations on encrypted data without exposing values to any software, developer, or IT operator.

Mojo-V enforces privacy entirely in hardware, rendering all software and developers untrusted. Every software-visible operation is either data-oblivious or cryptographically sealed within the processor.

Developed at the University of Michigan and Agita Labs, Mojo-V is the fifth generation in a lineage of secure architectures combining strong cryptography with secure hardware to advance system security and privacy:

Morpheus (I) → Morpheus II (II) → Sequestered Encryption Unit (III) → TrustForge Platform (IV) → Mojo-V (V)

2. Mojo-V Project Status

Mojo-V is an open project that is currently in development. **Figure 1** shows the projected timeline for the Mojo-V project. We are eager to receive input on this effort, so please feel free to send your feedback and contribute your own ideas. Here are additional details on the current Mojo-V project status:

- The current version of the Spike RISC-V ISA simulator with Mojo-V extensions is designed to be a ***complete reference implementation of Mojo-V***.
- Currently, Mojo-V is only specified as an extension to 64-bit RV64GC+Zicond RISC-V. (*Would an RV32 extension be desired?)*
- CSR register addresses and `lde`/`sde` and `flde`/`fsde` opcode assignments are not finalized as yet, as we will need to work with the RISC-V community to finalize these values.

- The hardware ciphers are not yet specified. Currently, we only specify the mechanism used to probe the hardware to see what ciphers it supports and the data contract field where the data owner can specify the ciphers they require to protect their data. We expect to codify this part of the Mojo-V specification once real hardware implementations start to materialize. As of this writing, the open-source project uses ***ML-KEM (Kyber)*** and ***Simon*** for its asymmetric and symmetric key ciphers, respectively.

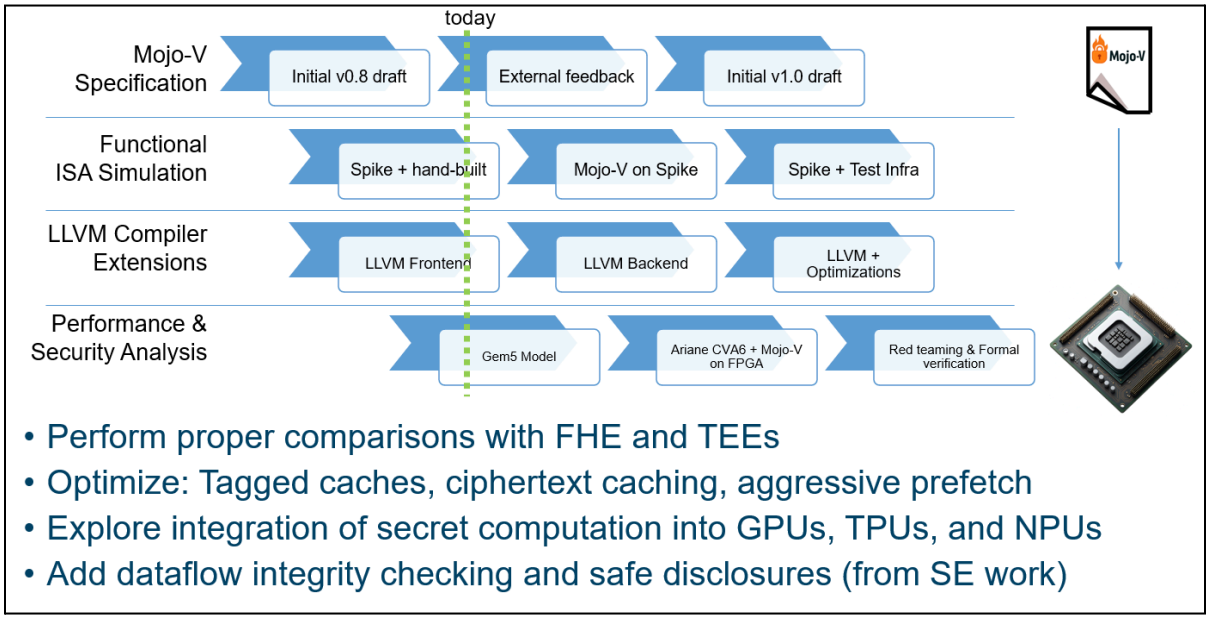

Figure 1: Mojo-V Timeline and Future Directions.

3. Secret Computation

As illustrated in **Figure 2**, Mojo-V enables privacy-preserving computation by ensuring that even untrusted cloud or edge infrastructure can process encrypted data safely and efficiently. Imagine a **smart-home security camera** that performs face and object recognition using an AI model hosted in the cloud.

Without Mojo-V, the cloud service must first decrypt each video frame to analyze it. The decrypted video is accessible to the service provider's software, developers, or IT staff, meaning privacy depends entirely on the provider's internal trustworthiness.

With Mojo-V, the camera encrypts each video frame using the data owner's key and transmits both the encrypted data and a *third-party encrypted data contract* to a Mojo-V-enabled cloud server. The Mojo-V processor decrypts and analyzes the data entirely **within hardware**, so unencrypted video is never visible to software or operators. Even a compromised operating system or malicious programmer cannot view, log, or leak the contents. With Mojo-V, **trust shifts completely into hardware**, removing the need to rely on software or developers for data privacy.

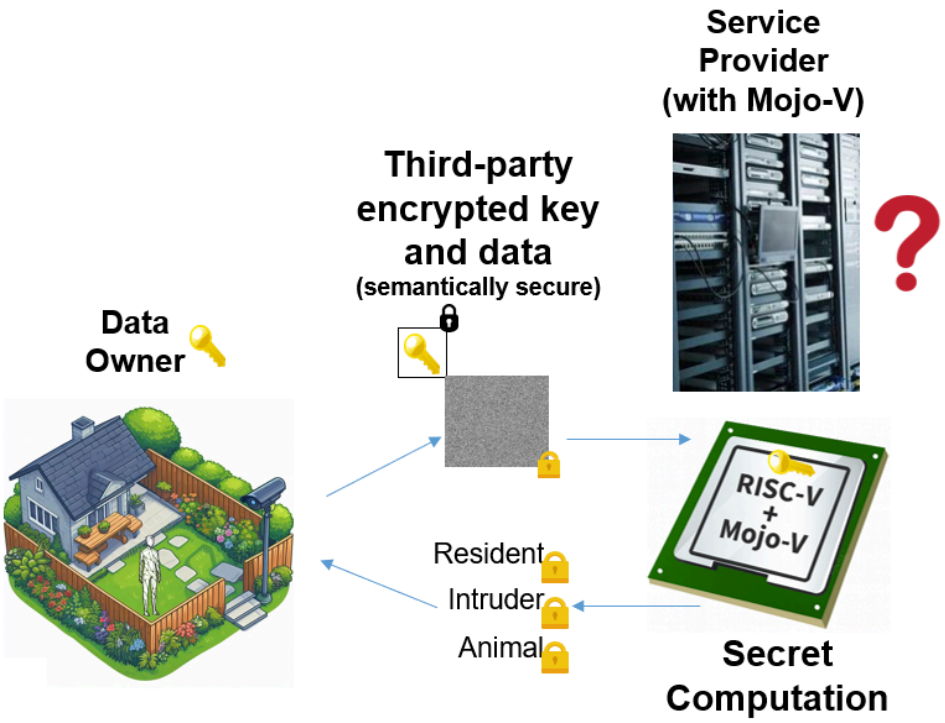

Figure 2: Privacy-Enhanced Cloud AI Application using Mojo-V.

Mojo-V implements **blind computation** through a hardware mechanism that isolates and controls the flow of secret data. It introduces the concept of **secret registers** and specialized semantics that strictly confine secrets within the processor. Protected data may only **enter** secret registers from a **third-party encrypted load**, and once marked secret, **every derived result** automatically inherits secrecy, forcing it back into a secret register. The **only legal path** for data to **leave** the secret registers via a **third-party encrypted store**, which re-encrypts the value under the data owner's key using strong, validated encryption. This design effectively **sequesters** secret values within hardware. Whenever secret register data is written to memory, it is encrypted under a key controlled by the data owner, never by the service provider. Contract-validated encryption ensures that any tampering or manipulation attempts are immediately detected by Mojo-V hardware during decryption.

Mojo-V also enforces **silent computation**, preventing any program behavior from revealing secret information. Hardware rules guarantee that program execution is completely data-oblivious — no secret value can influence control flow, operational timing, or memory access patterns. Three hardware-enforced restrictions make this possible:

1. **No secret registers in branch predicates** — prevents attackers from deducing secrets through timing differences, speculative execution, or instruction-path divergence.
2. **No secret code pointers or return addresses** — ensures that secret data cannot direct control flow or alter instruction fetch sequences, blocking instruction-cache and branch predictor side channels.

3. **No secret memory addresses** — prevents secret-dependent data memory references that could leak information through cache behavior, TLB lookups, or page faults.

These constraints eliminate observable differences between executions that depend on secret data. Every instruction sequence behaves identically from an external perspective, regardless of the secrets being processed.

Even under these restrictions, any Mojo-V computation must be expressed with a **data-oblivious programming model**. Traditional control flow and indexing can be restructured using conditional moves, oblivious lookup tables, or fixed-access patterns. The programmer writes normal code, while Mojo-V's ISA and compiler toolchain ensure that operations on secret data respect the enforced rules. Privacy is achieved automatically through hardware enforcement. Any program can be made to be data oblivious, and only code that computes on sensitive data requires this modification.

By combining **blind** and **silent** computation, Mojo-V establishes a framework for truly private, hardware-enforced computation. Secret data remains confined to secret registers and encrypted memory, while all observable software behavior remains independent of the secrets being processed. With Mojo-V, **privacy is guaranteed by silicon itself**, not by software policy or developer discipline.

4. Mojo-V Security Model

A **security model** defines what must be protected, who or what can be trusted, and how an attacker is expected to behave. It specifies the assumptions about the system, the capabilities and limits of adversaries, and what constitutes a successful breach. For Mojo-V, the model is intentionally conservative — granting attackers wide control over software and system resources while relying solely on hardware enforcement for secrecy and integrity.

Mojo-V follows a **zero-trust architecture**, assuming that all software — including operating systems, hypervisors, programmers, and administrators — may be compromised. Only Mojo-V hardware and the cryptographically secured data-owner channels are considered trustworthy. Table 1 depicts these trust boundaries, showing how Mojo-V isolates secret data inside hardware-protected domains while exposing only ciphertext to untrusted software layers.

In this model, only Mojo-V hardware may compute on decrypted data. All other entities interact exclusively with ciphertext. Any violation of this trust boundary triggers a Mojo-V security exception, terminating execution before a secret can be leaked.

Mojo-V's **assumptions** include a trusted hardware root of trust that is free of backdoors and design errors. Cryptographically secured communication channels ensure that only the data owner can install or refresh data contracts. A hardware-based true random number generator (TRNG) supplies salts for encryption freshness, while the physical silicon is assumed to be fabricated without malicious tampering.

Table 1. Mojo-V Security Model — Trust Boundaries.

Entity	Trust Level	Rationale
Software	Untrusted	Software compromise cannot reveal or alter protected data.
Programmers	Untrusted	Malicious application logic cannot intentionally or accidentally expose secrets.
IT / OS Staff	Untrusted	Administrative privileges provide no access to secret registers or encrypted memory.
Hardware (non-secret)	Untrusted	Non-Mojo-V hardware only handles ciphertext or public data.
Hardware (Mojo-V)	Trusted	Performs all secret computation on decrypted data inside sealed hardware registers with trusted data paths and ALU.

The **attacker** in Mojo-V's model is extraordinarily powerful. The adversary can control every layer of system software, execute arbitrary code at any privilege level, and even possess root access to servers. They may attempt side-channel analysis via operational timing, cache contention, branch prediction priming, or port contention analysis.

Mojo-V ensures that the attacker's influence ends at the hardware boundary. Attackers cannot read secret registers, extract plaintext through debug ports (e.g., JTAG), or forge ciphertext without valid keys and validation signatures. They also cannot manipulate ciphertext undetected, as contract-validated encryption ensures integrity and replay protection. As such, an **attack succeeds** only if an attacker achieves one of three outcomes:

1. **Secret disclosure** – obtaining plaintext or key material.
2. **Ciphertext Forgery** – introducing falsified ciphertext that passes validation.
3. **Replay compromise** – reusing old ciphertext to alter a computation.

Mojo-V's design prevents all three through hardware-enforced confidentiality, integrity, and freshness. Even with total software compromise, the system guarantees that secrets cannot be revealed, forged, or replayed. The result is a particularly powerful and resilient threat model — one that concedes near-total control to the attacker, yet still upholds the fundamental promise of Mojo-V: **no unwanted disclosures, even when every line of software is compromised**.

5. Mojo-V ISA Specification

This section defines the Mojo-V ISA extensions for secret computation on RISC-V.

5.1 Mojo-V Configuration CSRs

The Mojo-V Configuration Control and Status Registers (CSRs) define the architectural interface between hardware secrecy mechanisms and system-level software. These registers establish and maintain the cryptographic contract between the processor and the data owner, manage key installation, and track configuration state throughout secret computation. Together, they provide the necessary state control to enforce Mojo-V's zero-trust execution model, where secrecy is guaranteed entirely by hardware.

Table 2. Secret Configuration Registers.

CSR Name	Address	Access	Purpose
`mojov_cfg`	0x0A0	R/W	Controls secret configuration enable (`mojov_en`), key validity, encryption format, and zeroization state.
`mojov_ciphers`	0x0A1	Read-only	64-bit value indicating the asymmetric and symmetric key ciphers that are supported by this Mojo-V implementation.
`mojov_kmsm_addr`	0x0A2	R/W	Selects a byte address within the Key Management Shadow Map (KMSM).
`mojov_kmsm_data`	0x0A3	R/W	Transfers an 8-byte KMSM window beginning at `mojov_kmsm_addr`. Reads and writes are permitted at any valid 64-bit aligned byte address. Writes require kmsm_index[2:0] == 0.
`mojov_kmsm_ctrl`	0x0A4	R/W	Reports KMSM status and initiates contract opening through open_contract.

The **`mojov_cfg`** register is the central control register for Mojo-V secret computation. It governs whether the processor is operating in secret mode, indicates if a valid data contract is installed, and records the active encryption format and implementation version. It can be accessed and altered in any mode (M/S/U-mode) by any OS or application program. The `mojov_cfg` register contains the following bitfields:

Table 3. Bitfield Layout of the Mojo-V CSR Configuration Register. The `mojov_cfg` register controls activation of secret computation and tracks key validity, encryption format, and implementation version for Mojo-V's operation.

```
|31       12|11  4|3 2|1|0|
| Reserved | VER | FS|K|E|
```

Bit	Field	Abbrev	Access	Description
0	`mojov_en`	`E`	R/W	Enables secret computation mode.
1	`key_valid`	`K`	Read-only	Indicates a valid data contract is installed.
2-3	`format_sel`	`F`	Read-only	Active encrypted memory format (0 = fast, 1 = strong, proofcarrying = 2). Read-only.
4-11	`mojov_ver`	`VER`	Read-only	Contains an 8-bit version ID for the Mojo-V implementation, ensuring backward compatibility across revisions.

When **`mojov_en`** is asserted, Mojo-V enforces the secret computation rules described in Section 5.3, limiting data movement between secret and public domains. The **`key_valid`** bit signals that a legitimate data contract is active; operations involving secret data cannot proceed unless this bit is set. The **`format_sel`** field reflects which encryption format (fast, strong or proofcarrying) is active, determined by the data-owner's data contract. Finally, the **`mojov_ver`** field specifies the hardware implementation version of the Mojo-V ISA extension, allowing compilers and runtime systems to adapt to future revisions of the specification. Upon power-up or reset of the CPU, the **`mojov_en`** bit is initialized to zero, and Mojo-V secret computation is disabled.

The **`mojov_ciphers`** register is a read-only register that enumerates the asymmetric and symmetric ciphers supported by the current Mojo-V implementation. Each bit corresponds to a specific cipher algorithm; bits that are set to "1" indicate that the cipher is implemented and available for use by hardware cryptographic operations.

The register encodes two cipher groups:

- **Asymmetric key ciphers** — These bits identify public-key algorithms supported by the processor, such as **RSA-2048, ML-KEM-512,** and **Elliptic Curve P-256**.
- **Symmetric key ciphers** — These bits identify data encryption algorithms available for third-party encryption, such as **AES-128/256** and **Simon-128**.

Software or remote attestation tools may read **`mojov_ciphers`** to determine which cryptographic primitives are supported by a particular Mojo-V device before establishing a data contract or provisioning encryption keys. Each bit position is implementation-defined, and future revisions of Mojo-V may extend this register to indicate additional cipher suites or key-derivation mechanisms.

The **mojov_kmsm_addr** register is a read/write CSR that selects a byte address within the Key Management Shadow Map (KMSM). The KMSM is a CPU-internal, software-addressable staging structure used to exchange processor public-key material and encrypted contract-opening inputs with Mojo-V hardware. Full details of the KMSM are covered in Section 5.7 Key Management Interfaces. **mojov_msm_addr** may be accessed in M/S/U-mode. If **mojov_msm_addr** selects a byte outside the implemented KMSM range, the next access through **mojov_msm_data** fails and **mojov_kmsm_ctrl.status** is updated accordingly.

The **mojov_kmsm_data** register is a read/write CSR used to access KMSM contents. A read of kmsm_data returns the 64-bit value formed from bytes KMSM[**mojov_kmsm_addr** + 0] through KMSM[**mojov_kmsm_addr** + 7], interpreted according to the processor's natural endianness. Reads may be performed at any byte address for which all referenced bytes lie within the implemented KMSM range. A write to **mojov_msm_data** writes eight bytes to KMSM[**mojov_kmsm_addr** + 0] through KMSM[**mojov_kmsm_addr** + 7], interpreted according to the processor's natural endianness, but only if **mojov_kmsm_addr**[2:0] == 0. If **mojov_kmsm_addr** is not 64-bit aligned on a write, the write fails and no KMSM bytes are modified.

The **mojov_kmsm_ctrl** register is a read/write CSR that reports KMSM access status and initiates KMSM-controlled contract opening. Writing 1 to kmsm_ctrl.open_contract causes Mojo-V hardware to consume the staged ML-KEM ciphertext and staged encrypted data contract from KMSM, zeroize the writable KMSM staging regions, decapsulate the ciphertext using the processor-resident ML-KEM private key, decrypt the staged contract, validate its contents, and install the resulting contract into the active Mojo-V key state. If contract opening fails, the previously installed contract, if any, remains unchanged.

Table 4. Bitfield Layout of the Mojo-V CSR KMSM Control Register. The **mojov_kmsm_ctrl** register is a read/write CSR that reports KMSM access status and initiates KMSM-controlled contract opening.

```
|31        5|4 2|1|0|
| Reserved  | S  |B|O|
```

Bit	Field	Abbrev	Access	Description
0	open	O	Read-only	Writing a “1” initiates contract opening, using the encapsulated key and encrypted data contract information previously written into the KMSM.
1	busy	B	Read-only	Set while contract_open is in progress.
2:4	status	S	Read-only	Status result of most recent KMSM operation. 0 == success 1 == bad index 2 == alignment error

3 == decapsulation failure
4 == contract decryption failure
5 == contract validation failure

5.2 Mojo-V Exceptions

Mojo-V introduces a new architectural exception type known as the **Mojo-V Security Exception**, assigned to **exception code 0x1F** (reserved for architecture-specific security events). This exception is raised whenever a program attempts an operation that could disclose secret data or excite a side channel. Nearly all detection and enforcement occur at **decode time**, before any instruction is executed.

A Mojo-V Security Exception is triggered under the following conditions:

- Any instruction attempts to move or expose a secret value to a public register or unencrypted memory location.
- A secret value is used as a branch predicate, load/store address, or code pointer.
- A conversion or move operation (`fmv`, `fcvt`, or equivalent) attempts a **secret→non-secret** transfer.

This exception type allows Mojo-V hardware to prevent data leakage before it occurs, halting instruction issue for unsafe data flows. Because this enforcement happens during instruction decode, no secret-dependent execution ever takes place.

Additionally, secret-mode instructions **never raise runtime exceptions that would reveal a secret value**. Operations such as `div` or floating-point arithmetic that encounter exceptional conditions (e.g., divide-by-zero, underflow) must **drop the exception** and instead attach the condition as metadata to the resulting secret value. This ensures that secret data cannot influence observable program control flow or timing, preserving data-oblivious semantics throughout secret computation.

5.3 Secret Register Semantics

The **secret register semantics** ensure that values held in secret registers cannot be disclosed — neither directly through unencrypted stores or register moves, nor indirectly through timing, speculation, or any microarchitectural side channel. Secret registers have a single 64-bit data value that exists only within hardware; exceptions on secret operations are dropped rather than reported to software.

The Mojo-V architecture designates **x24–x31** and **f24–f31** as secret-capable registers. When secret mode is enabled (`mojov_en = 1`), these become the **secret general-purpose registers** (`p0–p7`) and **secret floating-point registers** (`pf0–pf7`) respectively. When secret mode is disabled, these registers revert to their standard RISC-V roles and are **automatically immediately zeroed** to prevent leakage of any secret data.

During instruction decode, the hardware verifies that secret and public operands are used safely. If any instruction violates these semantics, a *Mojo-V security exception* is raised immediately at decode. The rules are as follows:

- Any operation may compute on secret register values, but if any input operand is secret, then the destination register must also be secret.
- A secret register can be initialized through either an encrypted or non-encrypted load; however, third-party encrypted loads may only deposit their decrypted results into secret registers.
- Secret registers cannot be used as: (i) branch predicate inputs (e.g., `bne` operands), (ii) data address inputs (e.g., base registers for `ld` or `st`), or (iii) code pointer inputs (e.g., target address for `jalr`).
- The **`fmv`** and **`fcvt`** instructions, which transfer values between the integer and floating-point register files, are permitted only when data flow remains within the secret domain or when moving non-secret data into a secret register. Specifically, **secret→secret** and **non-secret→secret** conversions are allowed, while any **secret→non-secret** conversion will immediately trigger a *Mojo-V security exception* at decode time. Similar restrictions exist for the floating-point compare instructions, `feq.s, flt.s, fle.s, feq.d, flt.d, fle.d`. These instructions read from floating-point registers, but write the boolean comparison result to the integer register file.

The functional units that operate on secret registers must also ensure that timing behavior is independent of secret inputs. The system need not employ fully constant-time execution units; rather, it must guarantee that latency variations do not depend on secret data values. This requirement applies only to instructions marked as secret during decode.

When an instruction operating on a secret register encounters an exceptional condition (for example, divide-by-zero or floating-point underflow), the exception is dropped rather than reported to software. The hardware masks the event, preventing it from influencing observable control flow or timing. Dependent computations may continue using the resulting data value, ensuring that exception behavior cannot serve as a side channel.

When the OS triggers a context switch (via interrupt, exception, or ECALL) and Mojo-V secret registers are enabled, it must save and restore those registers only using Mojo-V's encrypted store/load instructions (SDE/FSDE and LDE/FLDE) to a per-process context area. The OS may move or buffer this ciphertext, but it never sees plaintext: the hardware refuses plaintext reads/writes of secret registers, and ciphertext is validated on restore. Any omission, replay, or tampering by the OS is detected during the encrypted reload or later when the data owner verifies results.

Finally, the only possible way for data to **exit** the secret registers is through a **third-party encrypted store**. In this operation, the value is encrypted under the data owner's key using signature-validated encryption before leaving the hardware boundary. This mechanism guarantees that secret data is never visible to software or developers and can only be shared

with authorized third-party entities through hardware-enforced, cryptographically protected channels.

Additional Secret Register Semantics Guidance

Secret register semantics have to be 100% correctly implemented, otherwise, attackers will find the hole in your security and through that hole will flow sensitive data. To aid in the implementation of secret register semantics, this section provides guidance for implementing these semantics for the RISC-V RV64GC+Zicond ISA as defined at the date of this specification's publication. (Note some additional instruction extensions are also included if relevant.)

- These instructions must restrict the use of secret registers as branch predicates: `beq, bge, bgeu, blt, bltu, bne, c_beqz, c_bnez`
- These instructions must restrict the use of secret registers as code pointers: `c_jalr, c_jr, jalr`
- These instructions must restrict the use of secret registers as load/store base pointers: `amoand.d, amoand.w, amocas.d, amocas.w, amomax.d, amomax.w, amomaxu.d, amomaxu.w, amomin.d, amomin.w, amominu.d, amominu.w, amoor.d, amoor.w, amoswap.d, amoswap.w, amoxor.d, amoxor.w, c_fld, c_fldsp, c_flw, c_flwsp, c_fsd, c_fsw, c_fswsp, c_lbu, c_ld, c_ldsp, c_lh, c_lhu, c_lw, c_sb, c_sd, c_sdsp, c_sh, c_sw, c_swsp, cbo.clean, cbo.flush, cbo.inval, cbo.zero, fld, flde, flw, fsd, fsde, fsw, lb, lbu, ld, lde, lh, lhu, lr.d, lr.w, lw, sc.d, sc.w, sh, sw`
- These instructions must drop their exception and status flag updates if executing an operation that writes its result to a secret register: `fadd.d, fadd.s, fcvt.d.l, fcvt.d.lu, fcvt.d.s, fcvt.d.w, fcvt.d.wu, fcvt.l.d, fcvt.l.s, fcvt.lu.d, fcvt.lu.s, fcvt.s.d, fcvt.s.l, fcvt.s.lu, fcvt.s.w, fcvt.s.wu, fcvt.w.d, fcvt.w.s, fcvt.wu.d, fcvt.wu.s, fdiv.d, fdiv.s, feq.d, feq.s, fle.d, fle.s, flt.d, flt.s, fltq.d, fltq.s, fmadd.d, fmadd.s, fmax.d, fmax.s, fmaxm.d, fmaxm.s, fmin.d, fmin.s, fminm.d, fminm.s, fmsub.d, fmsub.s, fmul.d, fmul.s, fnmadd.d, fnmadd.s, fnmsub.d, fnmsub.s, fround.d, fround.s, froundnx.d, froundnx.s, fsqrt.d, fsqrt.s, fsub.d, fsub.s`
- These instructions must not utilize secret inputs under any condition, because they write a potentially secret destination register plus a CSR register: `csrrs, csrrw, csrrc`
- These instructions, while they do not access memory directly, do manipulate the memory system in a way that could reveal secret register values, and thus, they cannot use secret register inputs under any condition: `hfence.gvma, hfence.vvma, sfence.vma`
- When CSR register `mojov_cfg` has a "0" written to its field `mojov_en`, Mojo-V secret computation is disabled and the implementation must immediately zeroize all secret architectural registers—`x24`–`x31` and `f24`–`f31`. This zeroization must occur before the CSR write retires so that no subsequent instruction, interrupt, or debug access can observe the prior contents of former secret registers. Afterward, reads of these registers

return zero as non-secret values. Additionally, upon power-up or reset of the CPU, the `mojov_en` bit is set to “0” and the secret registers must be cleared.

5.4 Third-Party Encrypted Loads and Stores

Third-party encrypted loads and stores are the **only** mechanisms by which sensitive data from an external data owner may enter or exit Mojo-V’s secret domain. These instructions enable secure cooperation between an untrusted service provider and a data owner, allowing encrypted computation to occur while ensuring that plaintext values are never visible to software, developers, or IT personnel.

The four instructions implementing this capability are **`lde`**/**`flde`** (Load Encrypted) and **`sde`**/**`fsde`** (Store Encrypted). All operate on ciphertext that conforms to a data contract provided by the data owner. The encryption employed is strong, semantically secure, and signature-validated through an **`auth_sig`** value embedded in the data contract. This guarantees that tampering, forgery, or replay of ciphertext values can be detected immediately by hardware.

Sensitive third-party data may be written to memory by an external device or process, provided the corresponding cryptographic key has been installed into Mojo-V via the `mojov_keycfg` CSR. Once installed, the LDE instruction can load this data into secret registers for computation, and SDE can store computation results back to memory, maintaining an end-to-end encrypted processing pipeline. At no point in this flow does unencrypted data appear in any software-visible register or memory location.

Encrypted Data Formats

Mojo-V supports multiple plaintext block formats for memory-resident ciphertexts, defined by the data owner’s data contract. A **fast** memory format provides more time/space efficiency at the expense of some security for third-party encrypted stores. The **strong** memory format provides significantly more security at the cost of increased storage and computation costs. The **proofcarrying** memory format incorporates the extra security of the strong-format encryption format, but also incorporates dataflow integrity checking, which attaches concise proofs of how a value was computed. The proofcarrying memory format also supports representations of datagrants, which are encrypted concise proofs that the `disc` and `fdisc` instructions utilize to disclose an encrypted value securely.

An attacker with access to a moderately powerful data center would be able to perform cryptanalysis on fast-format encrypted program values, likely being able to recover boolean values within a few days of analysis. Under strong-format or proofcarrying-format encryption, cryptanalysis is likely not possible with today’s data centers. Note that the memory format is specified by the data owner only, when their data control is installed (in the `format_sel` field). Service provider software that processes the third-party data is required to conform to the memory encryption format designated by the data owner.

Fast Encryption Format

```
struct {          // 128-bits in size, 128-bit alignment
  uint64_t val;           // register plaintext value
  uint32_t salt;          // random salt
  uint32_t auth_sig;      // validation signature (from contract)
} pt_fast;
```

The **fast** format optimizes speed over security. The plaintext 64-bit data value is stored in the `val` field. The field `auth_sig` is a data-owner–supplied constant inserted into the plaintext block of every third-party–encrypted value (32-bit in the fast format, 64-bit in the strong format). On `sde/fsde` instructions, hardware copies the contract's current `contract_sig` into the `auth_sig` field of the plaintext (alongside `val` and `salt`) before encrypting; on `lde/flde`, after decryption the hardware checks that the recovered `auth_sig` equals the one in the currently installed data contract, raising a Mojo-V Security Exception and discarding the data on any mismatch. Because stores use an avalanche cipher with fresh random 32-bit `salt`, any modification or cross-contract reuse of ciphertext produces effectively random plaintext, so a forged `auth_sig` matches only with negligible probability ($\approx 2^{-32}$ for fast format, 2^{-64} for strong format). The data owner can rotate `contract_sig` by reinstalling a new data contract via the control channel, instantly invalidating previously encrypted values and preventing replays.

Strong Encryption Format

```
struct {          // 256-bits in size, 128-bit alignment
  uint64_t val;           // register plaintext value
  uint64_t salt;          // random salt
  uint64_t auth_sig;      // validation signature (from contract)
  uint64_t metadata;      // target-specific metadata (e.g., device hash, overflow flag)
} pt_strong;
```

The **strong** format optimizes security over speed. It is similar to the fast format, except the random `salt` inserted into the encrypted packet is increased to 64-bits. Additionally, the `auth_sig` field is increased to 64-bits. Ciphertext forgery or cryptanalysis of strong-format ciphertext is expected to be computationally infeasible given a suitable strong cipher. The 64-bit metadata field is unused and can safely be filled with random salt if unneeded.

Proofcarrying Encryption Format

```
struct {          // 256-bits in size, 128-bit alignment
  uint64_t val;           // register plaintext value (or datagrant value)
  uint64_t salt;          // random salt
  uint64_t auth_sig;      // validation signature (from contract_sig, or contract_sig+1 for datagrant)
  uint64_t dfhash;        // dataflow hash (i.e., a concise proof for “val” or unused for datagrants)
} pt_proofcarrying;
```

The **proofcarrying** encryption format retains the security strength of the strong format, while also supporting proof-carrying computation. The `dfhash` field contains a hash signature that identifies the inputs and dataflow computation graph used to create the encrypted value in the `val` field. With a suitably strong cryptographic hash, it should be infeasible for an attacker to forge another computation dataflow graph with the same dataflow hash signature. More details of the dataflow hash generation process is detailed in Section 5.5.

The proofcarrying encryption format is also used to represent datagrant values. A datagrant is an encrypted dataflow hash value, which grants a computation that produces a value with that dataflow hash the ability to disclose that value (i.e., move it to a non-secret register). When the proofcarrying encryption format holds a datagrant, its `auth_sig` field is equal to `contract_sig + 1` and the `val` field is equal to the datagrant dataflow hash value. For datagrants, the `dfhash` field is unused.

SDE (Store Encrypted)

Instruction mnemonic:

```
sde rs2, offset(rs1)
```

Where:

- `rs1` – base register containing the memory address
- `rs2` – source register whose value will be encrypted and stored
- `offset` – 12-bit signed immediate (added to `rs1`)

Example asm code:

```
sde p3, 8(x10)
```

S-Type RISC-V instruction format:

```
|31      25|24-20|19-15|14-12|11      7|6-0|
|imm[11:5] | rs2 | rs1 |0x1  |imm[4:0] |0xb|
```

Instruction semantics: When a secret register value is written to memory using **`sde`**, the following occurs:

1. At decode time, Mojo-V verifies that secret mode is enabled (`mojov_en = 1` in CSR `mojov_cfg`) and that the source register (`rs2`) is a secret register; otherwise, a *Mojo-V security exception* is raised.
2. A new random salt is generated using the processor's true RNG.
3. The plaintext structure (fast or strong, depending on the data contract's `format_sel` field) is populated with the secret register value (`rs2`), true random salt, validation

signature from the current data contract (`contract_sig`), and any metadata fields defined by the data contract.
4. The plaintext block is encrypted using the symmetric key established through `mojov_keycfg`.
5. The resulting ciphertext is stored to the target memory address.

The encryption of the plaintext data must utilize a **block cipher with full avalanche effect**. As such, any changes in ciphertext will corrupt every bit of the plaintext and vice versa. This process ensures semantic security — identical plaintext values produce unrelated ciphertexts — and authenticity, where any modification to ciphertext causes decryption failure.

LDE (Load Encrypted)

Instruction mnemonic:

```
lde rd, offset(rs1)
```

Where:

- `rs1` – base register containing the memory address
- `rd` – destination register which will receive loaded and decrypted value
- `offset` – 12-bit signed immediate (added to `rs1`)

Example asm code:

```
lde p2, 16(x4)
```

I-Type RISC-V instruction format:

```
|31      20|19-15|14-12|11-7|6-0|
|imm[11:0] | rs1 |0x0   | rd |0xb|
```

Instruction semantics: When ciphertext is loaded from memory into a secret register using **`lde`**, the hardware performs the following steps:

1. At decode time, the processor verifies that secret mode is enabled (`mojov_en = 1` in CSR `mojov_cfg`) and that the destination (`rd`) is a secret register. Violations raise a *Mojo-V security exception*.
2. The ciphertext block is fetched from memory.
3. The block is decrypted using the symmetric key from the current data contract, which is active in the core's load/store encryption unit.
4. The plaintext block's signature (`auth_sig`) is compared against the current data contract validation signature (`contract_sig`). If a mismatch occurs, a security exception is raised and the data is discarded.
5. The plaintext's `val` field is written into the destination secret register (`rd`).

In both load and store paths, the `auth_sig` binds every ciphertext to the current data contract and cryptographic configuration (`contract_sig`). As a result, ciphertext encrypted under an older contract or a different data-owner key will fail contract validation when loaded.

If proofcarrying mode is enabled and the plaintext block's signature (`auth_sig`) is equal to (`contract_sig + 1`), this indicates that the load instruction is loading a datagrant value. The value is loaded into the register file, where it is marked in a metadata field as a datagrant value. Datagrant values can only be used by the `disc` and `fdisc` instructions as the third-party datagrant value. Any other use of the value, by any other RISC-V or Mojo-V instruction, results in a security exception.

FSDE (Floating-Point Store Encrypted)

Instruction mnemonic:

```
fsde frs2, offset(rs1)
```

Where:

- `rs1` – base register containing the memory address
- `frs2` – source floating-point register whose value will be encrypted and stored
- `offset` – 12-bit signed immediate (added to `rs1`)

Example asm code:

```
fsde f28, 8(x10)
```

S-Type RISC-V instruction format:

```
|31      25|24-20|19-15|14-12|11      7|6-0|
|imm[11:5] | rs2 | rs1 |0x3  |imm[4:0] |0xb|
```

Instruction semantics: When a secret register value is written to memory using **`fsde`**, the following occurs:

1. At decode time, Mojo-V verifies that secret mode is enabled (`mojov_en = 1` in CSR `mojov_cfg`) and that the source register (`frs2`) is a secret register; otherwise, a *Mojo-V security exception* is raised.
2. A new random salt is generated using the processor's true RNG.
3. The plaintext structure (fast or strong, depending on the data contract's `format_sel` field) is populated with the secret register value (`frs2`), true random salt, validation signature from the current data contract (`contract_sig`), and any metadata fields defined by the data contract.

4. The plaintext block is encrypted using the symmetric key established through `mojov_keycfg`.
5. The resulting ciphertext is stored to the target memory address.

The encryption of the plaintext data must utilize a **block cipher with full avalanche effect**. As such, any changes in ciphertext will corrupt every bit of the plaintext and vice versa. This process ensures semantic security — identical plaintext values produce unrelated ciphertexts — and authenticity, where any modification to ciphertext causes decryption failure.

FLDE (Floating-Point Load Encrypted)

Instruction mnemonic:

```
flde frd, offset(rs1)
```

Where:

- `rs1` – base register containing the memory address
- `frd` – destination floating-point register which will receive loaded and decrypted value
- `offset` – 12-bit signed immediate (added to `rs1`)

Example asm code:

```
flde f29, 16(x4)
```

I-Type RISC-V instruction format:

```
|31      20|19-15|14-12|11-7|6-0|
|imm[11:0] | rs1 |0x2  | rd |0xb|
```

Instruction semantics: When ciphertext is loaded from memory into a secret register using **`flde`**, the hardware performs the following steps:

1. At decode time, the processor verifies that secret mode is enabled (`mojov_en = 1` in CSR `mojov_cfg`) and that the destination (`frd`) is a secret register. Violations raise a *Mojo-V security exception*.
2. The ciphertext block is fetched from memory.
3. The block is decrypted using the symmetric key from the current data contract, which is active in the core's load/store encryption unit.
4. The plaintext block's signature (`auth_sig`) is compared against the current data contract validation signature (`contract_sig`). If a mismatch occurs, a security exception is raised and the data is discarded.
5. The plaintext's `val` field is written into the destination secret floating-point register (`frd`).

In both load and store paths, the `auth_sig` binds every ciphertext to the current data contract and cryptographic configuration. As a result, ciphertext encrypted under an older contract or a different data-owner key will fail validation when loaded.

Through the combination of `lde, sde, flde,` and `fsde` instructions, Mojo-V implements a closed cryptographic datapath where sensitive data can only enter and exit the processor through **third-party contract-validated encryption**. This mechanism enforces end-to-end confidentiality, integrity, and freshness while eliminating the need to trust any software layer in the computation chain.

5.5 Computational Integrity Checking with Dataflow Hashing

Dataflow Hashing

Mojo-V's core design protects **confidentiality**: secret values live only in secret registers and third-party encrypted memory, and no software can observe plaintext. However, confidentiality alone is not enough. A malicious or buggy service can still compute on ciphertext in *incorrect* ways—dropping inputs, fabricating outputs, or skipping parts of the computation—without ever seeing the plaintext and without violating encryption/validation rules. Mojo-V's integrity extension addresses this gap by attaching a verifiable, cryptographic "receipt" to every result value. Mojo-V's integrity checking is based on the "computational fingerprinting" technology detailed in this document.

To illustrate why integrity checking is important, consider a **Mojo-V-protected voting machine**. Each ballot is encrypted under the election authority's key and processed by a Mojo-V CPU in a hostile environment (e.g., an untrusted data center). Today's Mojo-V confidentiality rules ensure that no developer, operator, or OS can see individual votes. However, an attacker who controls the software could still:

- discard an encrypted vote instead of adding it to the tally, and
- fabricate a new encrypted tally that simply adds `+1` to their favored candidate.

Contract-validated encryption guarantees that ciphertext is not corrupted *in transit*, but it does *not* guarantee that the correct algorithm was executed on the correct inputs. The system needs a way for the data owner (the election authority) to verify that the published tally actually came from the approved computation over the approved ballots.

The **proof-carrying encryption** mode of Mojo-V provides this guarantee. Each time a program computes a value in secret registers, hardware maintains a compact **dataflow hash** that summarizes *how* that value was produced. Intuitively, every result carries a cryptographic "receipt" for the computation that produced it: a 64-bit hash value derived from a Merkle-tree hashing of the program's dynamic dataflow graph. When a result is eventually stored with the proof-carrying encryption format, this receipt is written into the `dfhash` field alongside the value, salt, and validation signature. The data owner (or another trusted entity) can later recompute the expected hash offline and check that the returned result's hash matches that expectation.

This mechanism builds on the data-oblivious nature of Mojo-V computation. Mojo-V forbids secret data from influencing control flow, memory addresses, or timing. As a result, the **dataflow graph** of a Mojo-V program—the graph of instructions and their data dependencies—is fixed for a given binary and non-secret environment. Changing secret inputs (e.g., different ballots) changes the values but **not** the shape of the dataflow graph. Therefore, the "expected" dataflow hash for a given program output is *independent* of the particular secret inputs or subsequent computation intermediate values; any run that follows the same dataflow graph yields the same dataflow hash.

To ensure that **third-party inputs are actually used correctly**, the data owner supplies **brands** when constructing encrypted inputs. A brand is a high-entropy 64-bit value written into the `dfhash` field of each third-party encrypted input packet. When the program performs an `lde`/`flde` on that packet, the Mojo-V core loads both the secret value and its brand; that brand acts as the root label for the leaf of the Merkle-tree hash computation. As the program runs, every instruction combines its inputs' labels into a new label, so the final output's dataflow hash can only be correct if:

- the *right* encrypted inputs (with the correct brands) were loaded, and
- those inputs were used at the *right points* in the computation, and
- the computation dataflow graph was faithfully executed.

Security of the scheme rests on the **computational hardness of finding collisions** in the hash function used to build the dataflow hash. An attacker who wants to "fake" a correct-looking result would have to construct a *different* dataflow graph—dropping or altering some inputs, or inserting extra operations—that nevertheless produces the same Merkle-root hash as the honest graph. For a well-designed hash, such collisions are computationally infeasible.

Dataflow Hash Implementation Details

If proof-carrying encryption mode is enabled by the data owner, Mojo-V computation will implement proof-carrying computation. Mojo-V's integrity extension is intentionally **transparent to software**. No new instructions are introduced; instead, the hardware tracks a hidden 64-bit **dataflow hash label** alongside each secret register value and uses the proof-carrying encryption format's `dfhash` field to store hashes in third-party encrypted memory.

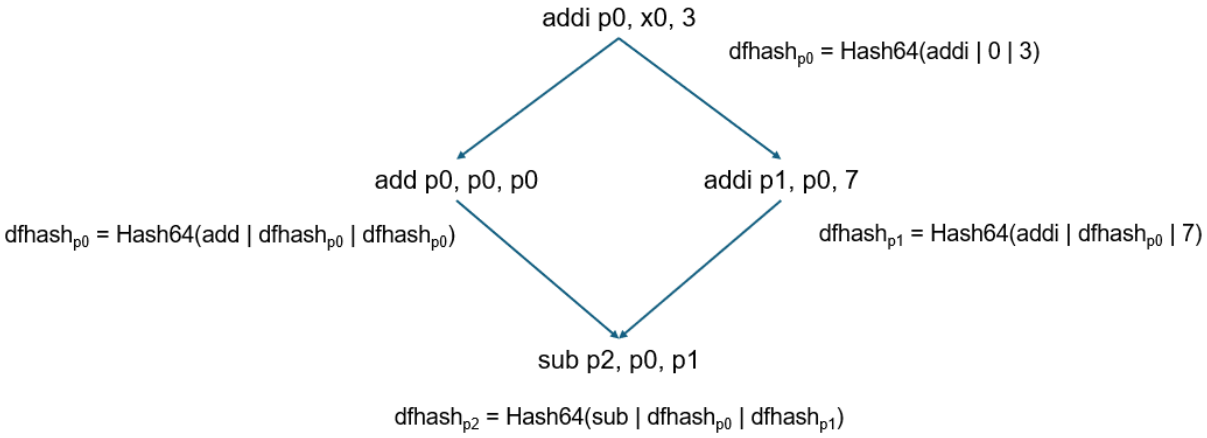

Figure 3: Example Dataflow Hash Computation.

At a high level:

- Each secret architectural register (p0–p7, pf0–pf7) has an associated hidden 64-bit metadata storage that stores the current dataflow hash for the value in that register.
- Every instruction that writes a secret destination register computes a new label from its opcode and its input operands' labels and/or values.
- `lde`/`flde` instructions initialize labels by loading the hash value from the `dfhash` field of third-party encrypted inputs. For data-owner encrypted third-party data, the data owner fills in the `dfhash` field with the input data **brand**, which is a leaf node hash label that allows data owners to specifically indicate where their inputs are to be incorporated into the expected dataflow computation.
- `sde`/`fsde` instructions export labels by writing the current label of the source secret register into the `dfhash` field of the proof-carrying plaintext block before encryption. Any encrypted value in memory can be returned to the data owner, where they can decrypt the value and inspect the dataflow hash used to compute the decrypted value.

Figure 3 illustrates an example dataflow hash computation for a simple four-instruction execution. The label update rule for a single instruction is as follows.

1. **Build a 64-bit opcode descriptor.**
   For each retired instruction that writes a secret destination register, Mojo-V constructs a 64-bit descriptor word that captures "what operation is being performed":
   - The **upper 32 bits** encode a mask of the instruction bits that participate in decoding this opcode (i.e., which bit positions are actually inspected by the decode logic).
   - The **lower 32 bits** pack the *values* of those opcode bits **plus all immediate fields** for the instruction (e.g., `IMM`, `shamt`, etc.). Register specifiers are *not* included, and for `lde`/`flde`/`sde`/`fsde` the base-register offset is not included in

the descriptor. The only immediate values included are those that affect the computation being produced by the dataflow graph.

This encoding ensures that only semantically relevant parts of the instruction word influence the hash, while register allocation and encrypted memory layout remain irrelevant.

2. **Collect input label words.**
   For each input operand:
   - If the operand is a **secret register**, the hardware takes its current 64-bit dataflow hash label, stored alongside the secret register as metadata.
   - If the operand is a **non-secret register**, the hardware uses the 64-bit value of that operand directly as its dataflow hash label. This ensures that any public constants which flow into the secret computation remain unchanged.

   This design has two effects: secret dependencies are tracked via their evolving labels, while non-secret contributions are “baked in” through their numeric values.

3. **Compute the result label.**
   The instruction’s output dataflow hash is computed as:

   ```
   dfhash = Hash64(opcode_word, [input_label0,] [input_label1,] [input_label2])
   ```

   where `opcode_word` is the 64-bit descriptor from Step 1, and $\texttt{input\_label}_i$ are the label words from Step 2 (with any commutative combination from Step 3 applied). The result label is written into the hidden label slot for the destination secret register in lockstep with the value writeback.

4. **Labels at third-party encryption boundaries.**
   - **Encrypted loads (`lde`/`flde`).** When loading a third-party encrypted value in the **proof-carrying format**, the decrypted plaintext provides both `val` and `dfhash`. The hardware writes `val` into the destination secret register and installs the decrypted `dfhash` as that register’s initial label. The data owner thus controls the **brand** associated with each external input.
   - **Encrypted stores (`sde`/`fsde`).** When writing a secret register to memory in proofcarrying format, the hardware builds the plaintext structure as usual (value, salt, auth_sig), and additionally writes the current label of the source secret register into the plaintext’s `dfhash` field before encryption. This label is the Merkle-root hash summarizing the computation that produced that value.

5. **Interaction with data-oblivious execution.**
   Because Mojo-V forbids secrets in branch predicates, memory addresses, and code pointers, the set of instructions contributing to a given secret value is independent of the

secret inputs. As a result, for any fixed program binary and non-secret environment, the sequence of opcode descriptors (and input wiring pattern) that feeds into the dataflow hash computation is invariant across runs with different secret data. Thus, the final `dfhash` for a given result depends only on:

- the *structure* of the dataflow graph, and
- the data-owner-controlled brands on third-party inputs and any public constants.

This enables the data owner to precompute or cache the expected hash for a given program and output location and then verify that a remote Mojo-V service executed that program faithfully.

6. **Hash function choice.**
   The **current Mojo-V implementation** uses a lightweight 64-bit mixing function, **SplitHash64**, as the core dataflow hash primitive. SplitHash64 is designed to be simple and efficient in hardware while providing good diffusion for typical workloads. Going forward, we expect Mojo-V implementations to adopt **stronger and more efficient hash constructions** (e.g., 128- or 256-bit cryptographic hashes with hardware acceleration) as they become available, further reducing the already small probability that an attacker could engineer a colliding dataflow graph.

Taken together, these mechanisms allow Mojo-V to **cryptographically bind each result to its computation history** without exposing any secret data or trusting the remote software stack. Dataflow hashing complements Mojo-V's confidentiality guarantees with verifiable integrity, enabling applications such as secure voting, regulated analytics, and cross-organizational computation where both secrecy and correctness of computation must be assured.

5.6 Safe Disclosures

If the proofcarrying memory mode is enabled, it is possible to implement a safe disclosure. A **safe disclosure** is a way to safely disclose an encrypted value, by moving it from a secret register to a non-secret register. A safe disclosure is only possible if a value computes a dataflow hash value that matches a datagrant dataflow hash value, supplied by the same third party that encrypted the data being used by the program. The datagrant indicates that the third-party owner has evaluated the dataflow computation uniquely named by the dataflow hash value in the datagrant, and that computation does not violate their view of data privacy.

As an example, if a Mojo-V program is processing encrypted data to train a machine learning model, the weights would be kept secret, and it would not be possible for the service provider to see those values. However, with a safe disclosure, it would be possible for the data owner to provide a datagrant to the service provider that would allow them to disclose the inference error over the last epoch of training, **provided that they computed the last epoch of training with correct computational integrity**. As such, safe data disclosures provide a highly flexible and secure mechanism for declassifying sensitive information. Safe disclosures are implemented with *i)* `lde` instructions that load datagrants, and *ii)* `disc` and `fdisc` instructions which enforce

valid disclosures. The semantics of the `lde` instruction when loading datagrants was detailed in Section 5.4. The `disc` and `fdisc` instructions are detailed below.

DISC (Integer Safe Disclosure)

Instruction mnemonic:

```
disc rd, rs1, rs2
```

Where:

- `rs1` – a secret register that will attempt a safe disclosure
- `rs2` – a datagrant value that will validate the safe disclosure
- `rd` – a non-secret destination register which will receive the disclosed value from `rs1`

Example asm code:

```
disc x2, x24, x25
```

R-Type RISC-V instruction format:

```
|31 25|24 20|19 15|14 12|11 7|6   0|
| 0x1 | rs2 | rs1 | 0x0 | rd | 0xb |
```

Instruction semantics: When a `disc` instruction attempts is made to disclose the secret value in `rs1` into the non-secret register `rd`, the hardware performs the following steps:

1. At decode time, the processor verifies that secret mode is enabled (`mojov_en = 1` in CSR `mojov_cfg`) and that the source #1 (`rs1`) is a secret register and destination (`rd`) is a non-secret register. Violations raise a *Mojo-V security exception*.
2. Source #2 (`rs2`) is verified to be a secret register and then its `datagrant` metadata is checked to ensure that it is a datagrant value. If either test fails, a *Mojo-V security exception* is raised.
3. The `concise proof` metadata of source #1 (`rs1`) is compared to the datagrant value in source #2 (`rs2`). If they match then the secret value in source #1 (`rs1`) is moved into the non-secret destination (`rd`) register. Otherwise, a *Mojo-V security exception* is raised.

FDISC (Floating-point Safe Disclosure)

Instruction mnemonic:

```
disc frd, frs1, rs2
```

Where:

- `frs1` – a secret floating-point register that will attempt a safe disclosure
- `rs2` – a datagrant value that will validate the safe disclosure
- `frd` – a non-secret destination floating-point register which will receive the disclosed value from `frs1`

Example asm code:

```
fdisc f2, f24, x25
```

R-Type RISC-V instruction format:

```
|31 25|24 20|19  15|14 12|11  7|6   0|
| 0x1 | rs2 | frs1 | 0x1 | frd | 0xb |
```

Instruction semantics: When a f`disc` instruction attempts is made to disclose the secret value in floating-point register `frs1` into the non-secret floating-point register `frd`, the hardware performs the following steps:

4. At decode time, the processor verifies that secret mode is enabled (`mojov_en = 1` in CSR `mojov_cfg`) and that the source #1 (`frs1`) is a secret register and destination (`frd`) is a non-secret register. Violations raise a *Mojo-V security exception*.
5. Source #2 (`rs2`) is verified to be a secret register and then its `datagrant` metadata is checked to ensure that it is a datagrant value. If either test fails, a *Mojo-V security exception* is raised.
6. The `concise proof` metadata of source #1 (`frs1`) is compared to the datagrant value in source #2 (`rs2`). If they match then the secret value in source #1 (`frs1`) is moved into the non-secret destination (`frd`) register. Otherwise, a *Mojo-V security exception* is raised.

5.7 Key Management Interfaces

The Mojo-V key management interface defines how cryptographic keys and **data contracts** are provisioned, maintained, and revoked within the processor. This mechanism forms the foundation for establishing trust between a data owner and the Mojo-V hardware during secret computation.

Mojo-V key management is anchored in hardware and operates entirely within the processor's trusted boundary. No software, including privileged system software or hypervisors, can view or modify plaintext key material. All key provisioning, installation, and verification occur through hardware-controlled interfaces that use the CSRs introduced in Section 5.1.

Key Management Shadow Map (KMSM, ML-KEM-512 variant)

The KMSM is a CPU-internal, software-addressable staging structure used to provision the processor's public encapsulation key and the encrypted inputs required to install a Mojo-V data contract. The KMSM is byte-addressable and is accessed through the CSRs kmsm_index, kmsm_data, and kmsm_ctrl. Reads of kmsm_data return an 8-byte window beginning at kmsm_index, interpreted according to the processor's natural endianness. Writes of kmsm_data update eight bytes beginning at kmsm_index, interpreted according to the processor's natural endianness, but are legal only when kmsm_index is 64-bit aligned.

Field Name	KMSM Address	R/W Permission	Field Size (in bytes)	Field Description
`pubkey`	0-799	Read-only	800 bytes	ML-KEM-512 encapsulation key
`enckey`	800-1567	R/W	768 bytes	ML-KEM-512 encapsulated-key ciphertext
`encdc`	1568-1631	R/W	64 bytes	Encrypted Mojo-V data contract (encrypted by encapsulated key)
`resv`	1632+	Reserved	TBD	Reserved for future use

Data Contract Format. The **data contract** is a 512-bit structure provided by the data owner, encrypted under the processor's `mojov_pubkey`, and installed into the hardware via the `mojov_keycfg` CSR. It defines the symmetric key, format, and validation parameters used during third-party encrypted computation. The structure aligns on 128-bit boundaries and is defined as:

```
typedef struct {
    uint64_t salt;          // random 64-bit
    uint8_t  sig[16];       // "Mojo-V ver. #001"
    uint8_t  sym_key_128[16];// 128-bit symmetric key (bytes, not C uint128_t)
    uint64_t contract_sig;   // unique 64-bit validation signature
    uint64_t ciphers;        // 64-bit mask (0 for now)
    uint8_t  format_sel;     // 0=fast, 1=strong, 2=proofcarrying
    uint8_t  pad[7];         // random padding to reach 64 bytes
} data_contract_t;
```

The **data contract** unifies key material, validation information, and encryption format selection into a single secure payload. The `contract_sig` value is a signature inserted into all third-party encrypted data, ensuring two security guarantees: *i)* the ciphertext has not been manipulated or forged, and *ii)* no replay of old ciphertexts has occurred since the last data contract installation.

The `ciphers` field of the data contract indicates the asymmetric and symmetric key ciphers that the data owner wishes to use. Only two bits can be set, for one asymmetric and symmetric key cipher. If *i)* more or less bits are set, or *ii)* the hardware doesn't support either of the requested

ciphers, then the data contract will not be successfully installed. The format of this field is the same as the format of the Mojo-V `mojov_ciphers` CSR register.

KMSM contract opening. When software writes 1 to `mojov_kmsm_ctrl.open_contract`, Mojo-V hardware performs the following steps:

1. Verify that `kmsm_ctrl.busy == 0`. If `busy == 1`, the request fails and no contract state is modified.

2. Set `mojov_kmsm_ctrl.busy = 1`.

3. Read the staged ML-KEM-512 encapsulated-key ciphertext from KMSM bytes 800-1567 and the staged encrypted data contract from KMSM bytes 1568-1631 into internal transient state.

4. Zero'ize the writable KMSM staging regions. This zeroization occurs for every accepted contract_open attempt, regardless of whether later decapsulation, decryption, or validation succeeds. The read-only processor public-key region is not modified.

5. Verify that `mojov_cfg.mojov_en == 0`. If secret computation is enabled, the request fails and no active contract state is modified.

6. Decapsulate the staged ciphertext using the processor-resident ML-KEM private key.

7. Decrypt the staged data contract using the key recovered by decapsulation.

8. Validate the resulting plaintext contract: i) the fixed signature string in `contract_sig`, ii) the requested cipher selections against `mojov_ciphers`, iii) the requested `format_sel` value, and iv) any additional implementation-defined consistency checks.

9. If validation succeeds, install the resulting contract into the active Mojo-V key state. Architecturally, this sets `mojov_cfg.key_valid`, updates `mojov_cfg.format_sel`, and loads the contract's symmetric key and `contract_sig` into the core's third-party encryption machinery.

10. If any step after acceptance fails, the previously installed contract, if any, remains active and unchanged.

11. Update `mojov_kmsm_ctrl.status` to reflect success or failure, then clear busy.

Key Provisioning and Installation. A data owner first reads the processor's ML-KEM encapsulation key from the read-only KMSM public-key region. The data owner then generates

an ML-KEM-512 encapsulated-key ciphertext and a shared secret, encrypts a 64-byte Mojo-V data contract under that shared secret, writes the ciphertext and encrypted contract into the writable KMSM regions, and requests installation by writing 1 to `mojov_kmsm_ctrl.open_contract`. Mojo-V hardware then consumes the staged ciphertext and encrypted contract, zero'izes the writable KMSM staging regions, decapsulates the staged ciphertext, decrypts and validates the staged contract, and, on success, makes that contract the active contract for third-party encrypted computation.

Security properties of KMSM access. KMSM access does not weaken Mojo-V's zero-trust model. Software can read only the processor's public encapsulation key and can write only ciphertext staging inputs. The ML-KEM private key, the decapsulated shared secret, the decrypted data contract, and the installed third-party symmetric key remain entirely within Mojo-V hardware and are never exposed through software-visible CSRs, registers, memory, or debug interfaces.

Key revocation and freshness. The data owner retains exclusive control of key rotation and revocation. When a data contract is revoked or updated, a new `auth_sig` value is installed to invalidate all previously encrypted data and prevent replay attacks. This ensures that only data encrypted under the current contract can be successfully loaded and decrypted.

Future extensions. A future revision of this specification will expand this section to define the detailed cryptographic protocol for:

- Public key attestation and trust establishment.
- Support for multi-key privacy-oriented applications.
- Hardware-based key derivation for sub-domains and multi-party computation.

These enhancements will further strengthen Mojo-V's capability to operate securely across distributed and multi-tenant environments while maintaining its principle of zero software trust.

6. Microarchitectural Considerations

Mojo-V's architecture enforces secrecy and data-oblivious behavior not only through its instruction semantics but also through carefully constrained microarchitectural design. All secret computation rules are applied at the earliest possible stage of execution, ensuring that no transient behavior, speculation, or optimization can leak secret information.

Decode-Time Enforcement. Security enforcement begins at instruction decode. Each instruction is analyzed to determine whether its operands or destinations involve secret registers. The decode stage ensures that secret and non-secret operands are never mixed in unsafe ways, and any violation — such as a secret register used in a branch predicate, load/store address, or CSR modification — immediately triggers a *Mojo-V security exception*. If any of the remaining rules apply, the decode stage also **propagates a single bit that indicates whether the instruction is secret or public**, based on the destination register. Because

operand secrecy is known statically at decode, this enforcement is deterministic and leaves no ambiguity for later pipeline stages.

Constant-Latency Functional Units. Mojo-V prevents timing-based information leaks by ensuring that operations involving secret operands execute with timing that is **independent of secret inputs**. Functional units need not be truly constant-time; they must simply ensure that execution latency does not vary with secret data. This constraint applies only to operations explicitly marked as secret (based on the decode-stage secrecy bit). This approach balances performance and security by isolating timing protections to instructions that actually process secret data.

Speculation Safety. Unconstrained speculative execution is permitted in Mojo-V, but all speculation involving secret data is constrained in one easy-to-identify condition. Wherever dependencies are speculatively inserted — primarily within the load/store queue — secret-to-non-secret speculative forwarding must be prohibited, as it could permit Spectre-style leakage of secret data into observable non-secret speculative register state. Secret-to-secret, nonsecret-to-secret, and nonsecret-to-nonsecret speculative forwarding are allowed, since they do not expose secrets across trust boundaries. These checks are simple to implement since the decode stage marks every operation unambiguously as secret or non-secret. This model preserves the performance benefits of speculation while preventing secret data from contaminating untrusted program state.

Exception Masking. When instructions involving secret operands trigger exceptions (for example, divide-by-zero or invalid operation), Mojo-V masks those exceptions so that no information about the secret operands is revealed. Masked exceptions neither raise traps to software nor alter observable architectural state. Instead, they are dropped or they propagate an exception metadata bit internally, maintaining silent and data-oblivious behavior. This ensures that secret-related faults cannot act as a side channel.

Compatibility and Integration Requirements. To maintain portability and predictable semantics, Mojo-V remains compliant with the standard RV64GC+Zicond RISC-V instruction set. In addition, Mojo-V implementations must include the `Zicond` conditional-move extension to allow data-oblivious predication. To support compatibility across developments, the `mojov_ver` field of the `mojov_cfg` CSR register includes an 8-bit version number that reflects the precise Mojo-V semantics implemented by the processor. This enables future revisions of the architecture to evolve while preserving backward compatibility and verifiable behavior.

7. System Software Considerations

This section details a number of system software considerations that must be addressed to fully utilize Mojo-V capabilities.

Compiler Support: The Mojo-V project is developing a Mojo-V capable version of the LLVM compiler. The compiler explicitly supports Mojo-V's **secret computation model** by allowing

programmers to declare variables and computations as secret. By convention, languages should express this using a `secret` attribute or type qualifier. When a secret variable is computed upon, the compiler must ensure that all intermediate values remain confined to **secret registers** (`p0–p7`, `pf0–pf7`). Any attempt to use a secret value as a control predicate, code pointer, or data pointer must trigger a compile-time error. The compiler should also expose a built-in conditional move intrinsic, such as `mojov_cmov()`, to allow side-channel–free predicated decision-making.

As an example of Mojo-V enabled code, please consider the code in Figure 4, which implements a data-oblivious version of bubble-sort for Mojo-V secret computation. The `data` variable is an array of secret integers. The bubble-sort algorithm iterates over the `size` integers in the array, performing a data-oblivious swap operation with the built-in compiler primitive `mojov_cmov`. `mojov_cmov` performs a data-oblivious conditional move: based on the first secret predicate operand, the primitive outputs the second (if true) or third (if false) operand. As this bubble-sort algorithm runs, the software and developers cannot see or infer the values of the `data` array, regardless of any coding changes or hacking of the bubble-sort software.

```
void bubblesort(secret int *data, unsigned size) {
  for (unsigned i = 0; i < size - 1; i++) {
    for (unsigned j = 0; j < size - 1; j++) {
      // swap needed?
      secret bool swap = (data[j] > data[j + 1]);

      // perform the swap
      secret int tmp = data[j];
      data[j]   = mojov_cmov(swap, data[j + 1], data[j]);
      data[j+1] = mojov_cmov(swap, tmp, data[j + 1]);

      // count the number of swaps executed
      swaps = mojov_cmov(swap, swaps + 1, swaps);
    }
  }
}
```

Figure 4: Mojo-V version of the Bubble-sort Algorithm.

There is **no trust placed in the compiler**—if it accidentally emits an instruction that would cause a side channel using a secret value, the Mojo-V hardware will detect this at decode time and raise a Mojo-V *security exception*.

Operating System Support: The operating system must preserve secret state across context switches. When secret mode is active (`mojov_en = 1`), the OS should save and restore secret registers using the **LDE** and **SDE** instructions. However, this does **not** imply trust in the OS—these operations occur entirely on encrypted data. Without OS support, an attempt to context-switch while in secret mode will result in a hardware Mojo-V s*ecurity exception* when the OS tries to save the secret registers (e.g., using `sd`), as Mojo-V forbids non-encrypted access to secret register values.

System Libraries: System libraries must be aware of Mojo-V's use of secret registers. Two key implications follow:

(a) **Compatibility builds.** Libraries that run on standard RISC-V systems may use all registers. When building a program with the `-fmojov` compiler flag, the toolchain should link against library versions that **do not** use the secret registers. This separation allows Mojo-V programs to reserve the secret register file without modification to library code. This register partitioning simplifies hardware, avoiding the need for data tagging, and allows all enforcement to occur cleanly at the decode stage.

(b) **Data-oblivious library variants.** Certain libraries—particularly `string`, `math`, and `sorting`—should include Mojo-V–enabled variants that use secret registers and implement **data-oblivious algorithms**. These functions can then safely operate on encrypted data while maintaining hardware-enforced privacy guarantees.

Client and Server Side Mojo-V Libraries: Both **data owners** (clients) and **service providers** (servers) benefit from Mojo-V software support libraries.

- **Client-side libraries** should include utilities for software-based key generation, encryption, and decryption, allowing data owners to prepare encrypted inputs and recover encrypted results.
- **Server-side libraries** should provide data-oblivious math and sorting routines, bindings for higher-level languages such as Python or JavaScript, and secure network marshalling functions to handle encrypted data exchanges.

Mojo-V Debug Support: Debugging in Mojo-V must preserve the same security principles as execution. The hardware must never expose decrypted data through debug interfaces. Instead, developers may use **software-based debugging modes** by inserting a **decrypted data contract** directly into the server environment. This enables debugging tools (such as GDB) to inspect data under development keys using software-based decryption algorithms. Production workloads using third-party keys cannot be debugged in plaintext—this is a feature, not a limitation, of Mojo-V's design. This guarantee ensures that even during development, production secrets remain inaccessible to software, administrators, or developers.

8. Authorship

Primary author: Todd Austin, University of Michigan / Agita Labs, austin@umich.edu

Additional contributors:
- Lauren Biernacki, Lafayette College
- Shibo Chen, Tenstorrent
- Max Christman, University of Michigan
- Meron Demissie, University of Michigan
- Yonathan Fisseha, University of Michigan
- Tarunesh Verma, University of Michigan

9. Change Log

Version 1.01 - June 15, 2026.

- Added support for safe disclosures (e.g., disc, fdisc instructions, datagrants)

Version 1.00 - March 16, 2026.

- Added Key Management Shadow Map (KMSM) facility for probing public key values and injecting encapsulated keys and encrypted data contracts.
- This release marks the first full implementation of (planned) Mojo-V capabilities in the RISC-V Spike instruction set simulator.

Version 0.93 - February 25, 2026.

- Further refined the key management interfaces, including the KMSM (key management shadow map).

Version 0.92 - November 26, 2025.

- Defined proofcarrying encryption support for Mojo-V that implements proof-carrying computation, used to perform remote computational integrity checking.
- Expanded INT and FP secret register set size (from 4 regs) to 8 regs.

Version 0.91 - November 10, 2025.

- Added FLDE and FSDE.
- Added more details on `contract_sig` vs. `auth_sig` validation signature handling.
- Renamed “weak” encryption mode to “fast”, and thus, now Mojo-V encryption modes are fast vs. strong.
- Added significantly more guidance on instruction-specific secret register semantics, including details on subtle restrictions, e.g., never secret inputs on CSR updates.
- Added a comparison to TEEs in Appendix C.

Version 0.90 – October 24, 2025.

- Initial release.

Appendix A. Comparison to FHE

Both **Fully Homomorphic Encryption (FHE)** and **Mojo-V** enable computation over encrypted data, but they differ fundamentally in their approach, performance, and security properties.

1. **Mathematical vs. Architectural Solutions**
   FHE is a **mathematical** solution to secret computation, built entirely upon cryptographic algebra and lattice-based security proofs. Mojo-V, by contrast, is an **architectural** solution—achieving secret computation by embedding secrecy enforcement directly into the processor hardware. In FHE, privacy derives from mathematical intractability; in Mojo-V, it derives from physical isolation and microarchitectural enforcement.
2. **Programming Experience and Performance**
   To programmers, both systems appear similar—computation proceeds directly on encrypted data, and a memory dump or software compromise reveals only ciphertext. However, Mojo-V executes using standard processor hardware and runs roughly **orders of magnitude faster** than a comparable CPU-based hybrid integer–Boolean FHE implementation. This makes Mojo-V the first architecture to deliver *interactive, human-scale secret computation* in real time.
3. **Attack Model and Threat Surface**
   From an attacker's perspective, FHE provides stronger theoretical protection, as it remains secure even against physical inspection. Mojo-V, on the other hand, is susceptible to **analog measurement attacks**—such as observing CPU power, thermal, or voltage fluctuations—that could potentially leak secret information. A key aspect of our follow-on work is to harden the plaintext portion of the Mojo-V hardware implementation against differential and direct analog measurement attacks, which will serve to narrow the security gap between Mojo-V and FHE.
4. **Data Representation and Programming Style**
   FHE systems generally offer two computational paradigms: **Integer FHE**, which is faster but difficult to express arbitrary programs in, and **Boolean FHE**, which is easier to program and fully Turing complete. Mojo-V's programming model is conceptually closer to Boolean FHE—supporting arbitrary control structures, loops, and data types—but operates at **near-native hardware speed**, vastly outperforming even optimized integer FHE platforms. Unlike FHE, Mojo-V supports standard floating-point modes.
5. **Roadmap and Future Integration**
   While Mojo-V does not rely on mathematical homomorphism, future versions will integrate **concise proofs** and **safe disclosures**—mechanisms similar to zero-knowledge proofs (ZKPs)—to allow verifiable results without revealing secrets. This will bridge Mojo-V's architectural secrecy with cryptographic proofs, providing hybrid trust assurance between physical enforcement and mathematical verifiability.

In summary, **FHE secures data through pure mathematics**, while **Mojo-V secures data through hardware semantics**. Both protect against software-level threats, but Mojo-V delivers

practical performance and programmability for consumer and enterprise workloads, complementing FHE's formal cryptographic strength.

Appendix B. Comparison to CHERI

The **CHERI (Capability Hardware Enhanced RISC Instructions)** architecture and **Mojo-V** share the goal of improving system trustworthiness through hardware support, but they approach the problem from fundamentally different perspectives.

CHERI extends a conventional RISC instruction set, compiler, and operating system to implement **fine-grained, capability-based memory protection**. Its primary goal is to mitigate **memory-safety vulnerabilities** in unsafe languages such as C by introducing unforgeable capabilities—pointers with explicit bounds and permissions—that prevent spatial and temporal memory errors. By combining these capabilities with operating-system-managed object compartments, CHERI provides **software compartmentalization** that can contain and limit damage from buffer overflows, pointer corruption, and similar exploit classes.

In contrast, Mojo-V ignores most vulnerabilities that CHERI tries to stop, and it instead **takes a data-centric approach**. Mojo-V does not attempt to eliminate software vulnerabilities; instead, it renders them harmless by protecting the *data* that attackers seek. Mojo-V uses **cryptographic isolation** and **hardware-enforced secret registers** to ensure that even if an attacker fully exploits a vulnerability, all retrieved information remains encrypted. While CHERI treats the *symptoms*—preventing corruption and unauthorized access through software correctness and pointer control—Mojo-V treats the *disease* by eliminating any value in the data exposed to attackers.

Conceptual Comparison:

Dimension	CHERI	Mojo-V
Security Objective	Prevents software vulnerabilities and enforces memory safety	Protects secret data from disclosure even in the presence of vulnerabilities
Primary Mechanism	Hardware-enforced capabilities for pointers and memory bounds	Hardware-enforced secrecy domain with encrypted storage and secret registers
Software Trust Model	Requires trusted compiler and OS to enforce capability semantics	Zero software trust; hardware enforces secrecy and integrity autonomously
Granularity of Protection	Per-pointer and per-object memory bounds	Per-register and per-memory-block cryptographic isolation

Failure Model	Compromise prevented by safety rules	Compromise tolerated but yields only ciphertext
Performance Impact	Low to moderate overhead for capability manipulation	Negligible overhead—executed at native CPU speed
Hardware Extension	Adds capability registers and tagged memory	Adds secret registers, encryption CSRs, and data-contract mechanism

Broader Security Impact: Mojo-V also mitigates a range of *microarchitectural* and *physical-layer* exploits that CHERI does not address. Because CHERI preserves the classic memory-based computation model, it remains susceptible to many side-channel and speculative-execution attacks that exploit timing, cache, or speculation leakage. Mojo-V, by contrast, **sequesters secret computation** and **encrypts all externalized state**, preventing data exposure from both digital and analog vectors.

Mojo-V mitigates attacks beyond CHERI's scope, including:

- Rowhammer
- Spectre V1/V2 and related speculative attacks
- Portsmash and Foreshadow
- Microarchitectural Data Sampling (MDS, RIDL, Zombieload, Fallout)
- Cold-boot and DMA extraction attacks
- PRIME+PROBE and other cache timing attacks
- Stack canary escapes
- GoFetch and prefetcher leakage
- Ciphertext analysis and CIPHERLEAKs
- Kocher and Bernstein timing side-channels
- Malicious developer or insider exfiltration
- And likely many *future, not-yet-invented* attack classes

Summary: CHERI provides a **software-centric solution** to software vulnerability mitigation—an evolutionary extension of the RISC model that enforces safer pointers and controlled compartmentalization through a trusted compiler and operating system. Mojo-V, in contrast, provides a **data-centric, cryptographically rooted solution** that assumes software may be compromised and instead ensures that no secret information can ever be exposed. In essence, **CHERI hardens code**, while **Mojo-V hardens data**, offering a simpler, faster, and more future-proof approach to computational privacy.

Appendix C. Comparison to TEEs

This appendix clarifies how Mojo-V differs from mainstream Trusted Execution Environments (TEEs, e.g., Intel SGX, AMD SEV/SEV-SNP, Intel TDX). It focuses on six dimensions that frequently cause confusion: what is attested, integrity of computation, cryptographic semantics,

practical exploit resistance, trust in programmers, and side-channel exposure. Where appropriate, we note exceptions or recent mitigations in some TEE variants.

1) What is attested?

TEEs. Remote attestation proves the initial identity and configuration of the protected container (enclave or VM). Concretely, reports bind secrets to a measurement of the initial image or launch state. Attestation does not prove that every subsequent step of execution followed a particular algorithm; it only authenticates the starting state.

Mojo-V. Mojo-V does not rely on trusting any software identity. Hardware enforces secrecy rules (secret registers, third-party encrypted loads/stores, and data-oblivious constraints) regardless of which software is running. The roadmap includes integrity extensions that will verify input provenance and dataflow integrity cryptographically.

2) Computation integrity

TEEs. After launch attestation, TEEs do not natively produce a proof that the computation was performed correctly. They protect isolation and, in some designs, memory integrity, but they do not emit verifiable proofs. Some TEEs (e.g., SGX) offer memory integrity/anti-replay, which differs from an externally verifiable proof that the computation followed a specific algorithm.

Mojo-V. Mojo-V will incorporate a computation-integrity mechanism: cryptographic verification of dataflow integrity and input provenance via per-packet metadata (e.g., Merkle-style hashing), without requiring trust in remote software or exposing secrets.

3) Cryptographic semantics of memory protection

TEEs. Many TEEs deploy deterministic, address-tweaked memory encryption to enable high-performance random access. Because ciphertext is visible and per-write randomization is limited, these schemes are typically not semantically secure (IND-CPA) for repeated writes at the same location, and ciphertext pattern analysis attacks have been demonstrated in practice.

Mojo-V. Mojo-V's third-party encrypted stores/loads (SDE/LDE, FSDE/FLDE) use contract-validated encryption with fresh salt and a data-owner supplied signature. Freshness and avalanche properties protects ciphertexts from being modified or forged, and per-value salt aims for semantic security of stored program values (especially under the strong format). Ciphertext visibility is a known risk in designs that trade stronger semantic guarantees for performance; Mojo-V's salted, validated encryption formats aim to close this gap.

4) "Makes hacking harder" vs. "stops data disclosures"

TEEs. TEEs harden systems but do not prevent exploitation outright. Enclave/guest bugs remain exploitable (e.g., ROP inside enclaves), and microarchitectural issues have bypassed

isolation in the past. Cloud verifiers typically rely on “I ran the measured binary,” not on a proof of each executed step.

Mojo-V. Mojo-V assumes software is untrusted (OS, hypervisor, applications, and developers). Secrets reside only in secret registers and leave the core only as contract-validated, salted ciphertext; unauthorized secret-to-public flows are blocked by the ISA’s decode-time checks.

5) Whom must you trust?

TEEs. TEEs primarily remove trust from the platform/OS/hypervisor and allow a relying party to pin trust to a measured binary. They do not remove trust from the application developer—bugs or malicious logic within the attested code can still leak via logic errors or side channels.

Mojo-V. Mojo-V removes trust from programmers. Secrecy is enforced by hardware rules: e.g., using a secret as a branch predicate/address/code pointer, or moving a secret to a public destination, raises a Mojo-V security exception.

6) Side-channel exposure

TEEs. TEEs do not eliminate side channels; attacks exploiting caches, page-fault patterns, speculation, or port contention have been repeatedly demonstrated, even when memory is encrypted. Mitigations exist, but the attack surface remains active.

Mojo-V. Mojo-V explicitly suppresses software-visible microarchitectural leakage by forbidding secrets in branch predicates, code pointers, and memory addresses and by masking exceptions on secret operations—enforcing data-oblivious behavior at the ISA boundary. (Physical-layer analog channels are out of scope for the base design and are future-work.)

Comparison Summary Table

Dimension	Typical TEEs	Mojo-V
What’s attested?	Initial code/data/config are measured and attested; proves the starting state, not step-by-step execution.	No software-identity trust required; secrecy rules are enforced in hardware; integrity proofs planned.
Computation integrity	No native proof of correct execution; isolation and (sometimes) memory integrity only.	Planned cryptographic dataflow/provenance verification via metadata/Merkle-style hashing.

Crypto semantics (memory)	Often deterministic address-tweaked encryption; ciphertext visibility enables pattern analysis in some settings.	Third-party contract-validated encryption with salt + auth_sig; strong format targets semantic security and replay detection.
Exploit resistance	Hardens but does not prevent exploitation (e.g., in-enclave ROP; past microarchitectural bypasses).	Assumes untrusted software; secret values never become software-visible, and illegal flows trap.
Programmer trust	Programmers remain trusted; bugs/malicious logic can leak via logic errors or side channels.	Trust removed from programmers; hardware rules enforce secrecy regardless of developer discipline.
Side channels	Numerous microarchitectural side channels remain an active threat despite mitigations.	ISA-level blind/silent rules suppress software-visible leakage; exceptions masked on secret ops.

Appendix C. Comparison to OISA

Mojo-V and OISA are both “data-oblivious execution” ISAs aimed at stopping *microarchitectural* leakage (speculation, timing, caches, predictors), and Mojo-V frankly **shares a lot of DNA with OISA**: both make *secret-bearing computation* run under strict rules, enforced by the hardware, so secrets can’t steer control-flow/memory behavior or leak via faults/optimizations.

What they’re each trying to accomplish

Mojo-V (secret computation on encrypted data): Extends the RISC-V RV64GC+Zicond ISA for “blind + silent” computation: secrets are never visible to software/operators, and execution is forced to be data-oblivious; secrets live in architecturally designated *secret registers* and enter/exit only through contract-validated encrypted loads/stores governed by a *data contract*.

OISA (a spec framework for data-oblivious execution on advanced μarches): Adds two abstractions—**Public vs Confidential** data labels and **Safe vs Unsafe** operands—to allow speculative/out-of-order cores while still meeting strong definitions like non-interference; Confidential data may only flow into Safe operands (where the hardware must suppress data-dependent side effects), otherwise the machine raises an exception before side effects occur.

The shared DNA (real overlap)

- **“Secrets must not affect observable behavior.”** Mojo-V states no secret in branch predicates, code pointers, or memory addresses; OISA captures the same idea via Safe/Unsafe operand rules—Unsafe operands cannot consume Confidential without triggering a label violation/fault.
- **Enforcement happens *before* leakage.** Mojo-V raises a Mojo-V Security Exception at decode time before execution; OISA raises a label violation immediately after operand read and before execute.
- **Faults/exceptions are treated as side channels.** Mojo-V masks/drops secret-triggered exceptions (or retains only internal metadata), to keep execution silent; OISA similarly requires handling (masking/suppressing) side effects and exceptions for Confidential→Safe uses, and forbids Confidential→Unsafe uses.
- **Speculation is not “turned off”; it’s made safe.** Mojo-V explicitly forbids secret→non-secret speculative forwarding; OISA’s local permission checks are designed to work regardless of whether instructions are speculative/out-of-order and avoid needing broad flushes in the common case.

Key differences (where they diverge sharply)

1) Confidentiality mechanism

- **Mojo-V:** Cryptographic confidentiality end-to-end—memory values are ciphertext; plaintext exists only inside the hardware secret domain, under a data-owner contract/key.
- **OISA:** Not “compute on ciphertext.” Data is tracked with **labels** (DIFT) and protected from *digital side channels*; for supervisor attackers it assumes a shielding system (e.g., SGX) prevents direct inspection/tampering. Developers and system operators have full access to sensitive values.

2) Trust model

- **Mojo-V: Zero-trust software** (OS/hypervisor/programmers/admins untrusted); only Mojo-V hardware + data-owner channels are trusted.
- **OISA:** Trusts the processor hardware and that the victim program uses OISA correctly; pairs naturally with enclaves for confidentiality against privileged software.

3) “Secret” representation and granularity

- **Mojo-V:** A small set of architecturally defined **secret registers** (x24–x31, f24–f31) plus special encrypted load/store pathways (LDE/SDE/FLDE/FSDE).
- **OISA: Word-granularity labels** tracked in RF + memory, plus **operand-level** Safe/Unsafe typing (more fine-grained than “this register is secret”).

4) Declassification / “getting results out”

- **Mojo-V:** Results typically leave as ciphertext via encrypted stores; plaintext never becomes software-visible by design.
- **OISA:** Supports explicit declassification via a *serializing* `ounseal` instruction (rare but necessary to return results).

5) Integrity

- **Mojo-V:** Today's spec foregrounds confidentiality, with contract-validated encryption and replay resistance via the data contract, and it also tracks input and dataflow graph computational integrity (e.g., provenance/dataflow).
- **OISA:** Explicitly *does not* target integrity of computation (orthogonal mechanisms).